COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures & Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:	09/08/2026	**DATE SUBMITTED:**	09/10/2026
NAME:	Baltazar, Kenji	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 7.1 **Hands-on Activity 7.1 Sorting Algorithms Pt1**

1. PROCEDURE OUTPUT

CODE

```
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "sortingAlgo.h"

void display(int arr[]);
const int maxSize = 100;

int main() {
    int unsortedArr[maxSize];
    srand(time(0));
    for (int i = 0; i < maxSize; i++) {
        unsortedArr[i] = rand() % 100 + 1;
    }
    std::cout<<"random numbers: \n";
    display(unsortedArr);

    std::cout<<"bubble sort: \n";
    bubbleSort(unsortedArr, maxSize);
    display(unsortedArr);
```

```
20      display(unsortedArr);
21
22      std::cout<<"insertion sort: \n";
23      insertionSort(unsortedArr, maxSize);
24      display(unsortedArr);
25
26      std::cout<<"selection sort: \n";
27      selectionSort(unsortedArr, maxSize);
28      display(unsortedArr);
29
30
31      return 0;
32  }
33
34  // FUNCTION DEFINITION/S
35  void display(int arr[]) {
36      for (int i = 0; i < maxSize; i++) {
37          std::cout << arr[i] << " ";
38      }
39      std::cout << std::endl;
40  }
```

OUTPUT

```
random numbers:
13 47 74 21 58 46 68 82 54 72 81 16 33 97 39 88 35 39 89 23 66 91 31 10 59 100 54 61 49 95 38 16 6 91 4
2 100 66 98 75 7 25 55 22 52 84 69 55 1 40 93 94 53 6 68 11 48 98 99 96 53 21 82 84 83 76 77 73 11 52 5
 63 89 14 72 79 17 60 68 57 8 49 34 42 75 87 36 4 75 4 47 83 22 49 95 63 5 47 87 83 98
```

OBSERVATION

This is the main function where every sorting algorithm is called, the main function starts of by using a random number generator as seen on line 11. Then uses a loop called **display** which is found in line 35, where its purpose is to display up to **maxSize** random numbers which in this case is 100. This is an O(n) worst & best time complexity as well as it only has one loop that depends on n/i.

Table 7-1. Array of Values for Sort Algorithm Testing

CODE

```
5   template <typename T>
6   void bubbleSort(T arr[], int maxSize){
7       for (int i = 0; i < maxSize; i++){
8           for (int j = i + 1; j < maxSize; j++){
9               if (arr[j] < arr[i]){
10                  std::swap(arr[j], arr[i]);
11              }
12          }
13      }
14  }
```

OUTPUT

```
bubble sort:
1 4 4 5 5 6 6 7 8 10 11 11 13 14 16 16 17 21 21 22 22 23 25 31 33 34 35 36 38 39 39 40 42 42 46 47 47 4
7 48 49 49 49 52 52 53 53 54 54 55 55 57 58 59 60 61 63 63 66 66 68 68 68 69 72 72 73 74 75 75 75 76 77
 79 81 82 82 83 83 83 84 84 87 87 88 89 89 91 91 93 94 95 95 96 97 98 98 98 99 100 100
```

OBSERVATION

This sorting algorithm is a bubble sort, where for every index i, it checks every element after it as seen in **j = i + 1**. And when it finds an element that is smaller than our current **arr[i]**, it swaps them as seen in the if statement code line. This inner loop will finish after checking every element and that the smallest element is now on the left most side. When the inner loop has finished executing the outer loop will then run the second time and run another inner loop, this will go on until all the element in the array has been sorted out from smallest to largest.
As seen on this bubble sort, its worst time complexity is $O(n^2)$ because it is a nested loop and from our analyzation, we can determine that it does a full check of all elements regardless if the smallest has been found or not, so our best time complexity is still $O(n^2)$.

Table 7-2. Bubble Sort Technique

CODE

```cpp
template <typename T>
int routine_smallest(T A[], int maxSize, int k){
    int position=k;
    T smallestElem = A[k];
    for (int J = k+1; J < maxSize; J++){
        if (A[J] < smallestElem){
            smallestElem = A[J];
            position = J;
        }
    }
    return position;
}
```

```cpp
template <typename T>
void selectionSort(T arr[], int arrSize){
    int POS;

    for(int i=0; i < arrSize; i++){
        POS = routine_smallest(arr, arrSize, i);
        std::swap(arr[i], arr[POS]);
    }
}
```

OUTPUT

```
selection sort:
1 4 4 5 5 6 6 7 8 10 11 11 13 14 16 16 17 21 21 22 22 23 25 31 33 34 35 36 38 39 39 40 42 42 46 47 47 4
7 48 49 49 49 52 52 53 53 54 54 55 55 57 58 59 60 61 63 63 66 66 68 68 68 69 72 72 73 74 75 75 75 76 77
 79 81 82 82 83 83 83 84 84 87 87 88 89 89 91 91 93 94 95 95 96 97 98 98 98 99 100 100
```

OBSERVATION

This sorting algorithm is a selection sort, where it uses the function called **routine_smallest**. This **routine_smallest** finds the smallest element and holds its element and position as seen on the if statement on line 37 but is still a subject of change whenever a smaller element is found. Then it returns that position to the **selectionSort** function in the for loop section where it swaps that value to index i as seen on line 52, this will go on until the array is sorted out. Then as usual, its worst time complexity is $O(n^2)$ as it is a nested loop, the for loop on the **selectionSort** is the outer loop while the **routine_smallest** is the inner loop. But there is no best time complexity as it is still $O(n^2)$ because there is no condition for the inner loop skip past because it will still check for every element and cannot determine if the value it holds is the smallest until it checks all the elements, even if the

first element element is the smallest.

Table 7-3. Selection Sort Algorithm

CODE

```
16  template <typename T>
17  void insertionSort(T arr[], int maxSize){
18      int k = 1, j;
19      T temp;
20
21      while (k < maxSize) {
22          temp = arr[k];
23          j = k-1;
24          while (j >= 0 && temp < arr[j]){
25              arr[j+1] = arr[j];
26              j--;
27          }
28          arr[j+1] = temp;
29          k++;
30      }
31  }
```

OUTPUT

```
insertion sort:
1 4 4 5 5 6 6 7 8 10 11 11 13 14 16 16 17 21 21 22 22 23 25 31 33 34 35 36 38 39 39 40 42 42 46 47 47 4
7 48 49 49 49 52 52 53 53 54 54 55 55 57 58 59 60 61 63 63 66 66 68 68 68 69 72 72 73 74 75 75 75 76 77
 79 81 82 82 83 83 83 84 84 87 87 88 89 89 91 91 93 94 95 95 96 97 98 98 98 99 100 100
```

OBSERVATION

This sorting algorithm is the insertion sort, where for each k starting from 1, it takes those current element as seen in the line 22, **temp = arr[k]** then hold those element while comparing to the elements before it, and it starts in **j=k-1**. And then seen on the inner loop, it first checks if j is smaller than **arr[j]** and if yes then it moves the **arr[j]** to the right due to line 25 and also moves j backward because we decrement, why? Because j is the one that checks for a spot where temp could go, if there is not decrement then j would just keep overwriting the same spot. Then once the inner loop is done executing, the temp is placed on to the gap created in the inner loop, this will go on until all is sorted out. Then as always, its worst time complexity is $O(n^2)$ because this is a nested loop, but the best

case is just an O(n) because if the condition of **j… < arr[j]** is not fulfilled, then the while loop will not execute and will run another outer loop instead.

Table 7-4. Insertion Sort Algorithm

2. ANSWER TO SUPPLEMENTARY ACTIVITY

PSEUDOCODE

```
START
   INITIALIZE maxSize = 100
   INITIALIZE numcrocodiles = 5
   INITIALIZE tallyVotes
   INITIALIZE winnerchickndinner

   INITIALIZE unsortedArr
   SEED random generator with current time
   FOR i = 0 TO maxSize - 1
      unsortedArr = random numbers from 1 to 5
   END LOOP

   INITIALIZE politician = [
      "Bo Dalton Capistrano",
      "Cornelius Raymon Agustin",
      "Deja Jayla Banaga",
      "Lalla Brielle Yabut",
      "Franklin Relano Castro"
   ]

   DISPLAY "All votes:"
   CALL display
   CALL insertionSort
   DISPLAY "Tallied votes:"
   CALL display
   CALL tallyVotes
   winnerIndex = CALL winnerChicknDinner

   DISPLAY "Vote Tally:"
   FOR c = 0 TO numCandidates - 1
     DISPLAY politician ": " voteCount " votes"
   END LOOP

   DISPLAY "Winner: " politician

END
```

```
FUNCTION display
   FOR i = 0 TO maxSize - 1
      DISPLAY arr[i], " "
   END LOOP
END FUNCTION

FUNCTION tallyVotes
   FOR i = 0 TO maxSize - 1
      voteCount = voteCount + 1
   END LOOP
END FUNCTION

FUNCTION winnerChicknDinner
   FOR c = 1 TO numCandidates - 1
      IF voteCount > voteCount THEN
         winner = c
      END IF
   END LOOP
   RETURN winner
END FUNCTION
```

CODE

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "sortingAlgo.h"

void display(int arr[]);
const int maxSize = 100;
const int numcrocodiles = 5;

void tallyVotes(int arr[], int maxSize, int voteCount[]);
int winnerchickndinner(int voteCount[]);

int main() {
    int unsortedArr[maxSize];
    srand(time(0));
    for (int i = 0; i < maxSize; i++) {
        unsortedArr[i] = rand() % numcrocodiles + 1;
    }
    std::cout << "All votes:\n";
    display(unsortedArr);

    int voteCount[numcrocodiles] ={0, 0, 0, 0, 0};
    std::string politician[numcrocodiles] ={
        "Bo Dalton Capistrano",
        "Cornelius Raymon Agustin",
```

```
        "Cornelius Raymon Agustin",
        "Deja Jayla Banaga",
        "Lalla Brielle Yabut",
        "Franklin Relano Castro"
    };
    insertionSort(unsortedArr, maxSize);

    std::cout << "\nTallied votes:\n";
    display(unsortedArr);

    tallyVotes(unsortedArr, maxSize, voteCount);

    int winnerIndex = winnerchickndinner(voteCount);
    std::cout << "\nVote Tally: \n";
    for (int c = 0; c < numcrocodiles; c++) {
        std::cout << politician[c] <<": " << voteCount[c] << " votes\n";
    }
    std::cout << "\nWinner: "<< politician[winnerIndex]<<"\n";

    return 0;
}

void display(int arr[]) {
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }
```

```
51      std::cout << std::endl;
52  }
53
54  void tallyVotes(int arr[], int maxSize, int voteCount[]) {
55      for (int i = 0; i < maxSize; i++) {
56          voteCount[arr[i] - 1]++;
57      }
58  }
59
60  int winnerchickndinner(int voteCount[]) {
61      int winner = 0;
62      for (int c = 1; c < numcrocodiles; c++) {
63          if (voteCount[c] > voteCount[winner]) {
64              winner = c;
65          }
66      }
67      return winner;
68  }
```

OUTPUT

TRIAL 1

```
All votes:
2 3 4 4 2 4 2 4 3 4 5 1 4 1 4 3 4 1 2 4 2 2 1 5 4 3 4 1 4 2 3 2 2 1 3 5 1 2 3 5 3 1 1 4 2 2 4 2 1 4 4 4 4 1 2 3 1 5 3 4 4
 2 4 4 2 4 3 3 4 1 5 3 4 5 3 1 5 1 5 5 3 3 5 2 5 4 2 3 3 5 1 5 5 3 5 1 1 2 3 1

Tallied votes:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4
 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5

Vote Tally:
Bo Dalton Capistrano: 19 votes
Cornelius Raymon Agustin: 19 votes
Deja Jayla Banaga: 20 votes
Lalla Brielle Yabut: 26 votes
Franklin Relano Castro: 16 votes

Winner: Lalla Brielle Yabut
```

TRIAL 2

```
All votes:
1 3 5 1 5 5 1 2 1 4 4 5 4 1 3 5 5 2 3 5 4 3 3 1 1 1 4 5 5 5 5 2 5 2 1 3 3 3 5 2 1 3 2 1 5 4 3 1 1 2 2 2 2 1 5 3 1 2 1 5 3
 3 3 1 2 2 2 2 1 1 2 3 1 4 3 2 4 4 2 3 3 2 1 3 3 4 4 2 4 4 2 4 5 2 1 2 1 1 4 3

Tallied votes:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3
 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5

Vote Tally:
Bo Dalton Capistrano: 24 votes
Cornelius Raymon Agustin: 23 votes
Deja Jayla Banaga: 21 votes
Lalla Brielle Yabut: 15 votes
Franklin Relano Castro: 17 votes

Winner: Bo Dalton Capistrano
```

TRIAL 3

```
All votes:
2 3 3 5 1 2 1 4 1 3 3 3 5 2 2 3 5 1 3 1 3 4 4 1 5 3 5 3 2 2 1 1 5 2 5 1 1 4 3 2 4 4 2 3 5 5 3 1 1 4 3 5 2 1 3 4 1 1 1 5 4
 3 2 4 3 4 5 5 3 5 3 3 5 3 5 5 5 1 5 5 4 3 3 1 3 5 5 4 2 3 1 1 5 4 5 5 4 1 4 3

Tallied votes:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4
 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5

Vote Tally:
Bo Dalton Capistrano: 21 votes
Cornelius Raymon Agustin: 12 votes
Deja Jayla Banaga: 26 votes
Lalla Brielle Yabut: 16 votes
Franklin Relano Castro: 25 votes

Winner: Deja Jayla Banaga
```

OBSERVATION

This is supplementary code, where it first generates the 100 random numbers from 1 to 5 as seen in line 17, then we sort them out using the insertion sort from the previous header file so that the votes can be sorted and grouped together. Then once everything is sorted out, **tallyVotes** goes through the array and checks the votes and increases vote count per votes. Then the **winnerChicknDinner** loops through the array and checks which index holds the highest number as seen in line 63. This algorithm has a worst time complexity of $O(n^2)$ as well as having an O(n) as best time complexity.

QUESTIONS

1. Why do you chose to use this sorting algorithm?
   - Unlike bubble and selection sort that has a $O(n^2)$ as a best time complexity, the insertion sort in this program has a best time complexity of O(n) because of the **while (j >= 0 && temp < arr[j])**, this specific line helps the code to not waste time when it hits an equal value especially as the random values in this code for voting only have 1 to 5 values.
2. Was your developed vote counting algorithm effective? Why or why not?
   - Yes ofcourse, because this vote counting algorithm can sort unsorted arrays in a O(n) time frame in its best case. Although it is still an $O(n^2)$ in its worst case, it is very efficient in small datas like this. The downside for this vote counting algorithm is that it is only best if it is in small datas but for big datas, it will take longer for this algorithm. So if it comes to that, the

insertion sort will be replaced to a much more efficient sorting methods like the merge sort, shell sort, or quick sort.

3. CONCLUSION

In this hands on activity, I learned the basic sorting algorithms: insertion, selection, bubble. I learned that bubble sort and selection sort both has a time complexity of $O(n^2)$ whilst the insertion sort can have a best time complexity of O(n) while having a worst time complexity of $O(n^2)$. I realized as well that insertion sort is much more efficient to use than bubble or selection but it has it limits and is really not suited for big data. For the procedures, implementing all those lines of codes helped me to understand these three different sorting algorithms, analyzing them also helped me to understand the basics and the concept behind it. In the supplementary, I was able to utilize my knowledge behind the insertion sort to make a vote counting algorithm. Overall, I think I did great in this activity as I understand the concepts behind the sorting algorithms well, though I want to improve on the coding itself as I am more verse in concept before the coding for now.

4. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude & ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Pseudocode and code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The extent of use of AI was just an assistance for the srand part of the pseudocode as well as assisting in the formatting for it. For the code, it helped by assisting in coding some functions related to the supplementary. But the completion of my work is my own, the analysis and the original thinking is from my own work.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.